Student Names- Sneha Saha, Shreya Tarafdar

Batch- 2022-2025

Institute- Asutosh College

Project Guide- Koustab Ghosh

Report submitted to : IDEAS- Institute of Data Engineering,Analytics and Science Foundation,

ISI Kolkata.

1. ABSTRACT

This project aims to develop a Movie Recommendation System using Python, using machine learning algorithms to suggest movies to users based on their preferences. The recommendation system will utilize content-based filtering techniques to provide personalized movie recommendations. It will recommend movies with similar features to those the user has liked in the past. The system will be built using popular Python libraries such as Pandas and Numpy for data manipulation, Scikit-learn for implementing machine learning algorithms and Streamlit for web app interface . This project will provide insights into the practical implementation of recommendation algorithms and their applications in real-world scenarios, enhancing the user experience by tailoring movie suggestions to individual tastes.

2.INTRODUCTION

In today's digital age, where vast amounts of data are generated daily, recommendation systems play a crucial role in enhancing user experience by providing personalized suggestions. One such application is the Movie Recommendation System, which aims to suggest movies tailored to individual preferences based on historical data and user interactions.

RELEVANCE AND IMPORTANCE

The relevance of a Movie Recommendation System lies in its ability to help users discover new movies they are likely to enjoy, thereby increasing user engagement and satisfaction on streaming platforms like Netflix, Amazon Prime, and others. By analyzing user preferences and behaviors, these systems can significantly improve content discovery, leading to longer user sessions and increased platform retention.

TECHNOLOGY INVOLVED

Developing a Movie Recommendation System involves using machine learning techniques such as content-based filtering. Python, with its rich ecosystem of libraries like Pandas, NumPy, Streamlit and Scikit-learn, provides robust tools for data preprocessing, algorithm implementation, and model evaluation.

BACKGROUND MATERIAL SURVEY

A variety of data are available on the web. Sources like kaggle and IMDb websites are equally trustworthy and efficient, after conducting a through search and diligently judging the data, we've concluded upon selecting the movies dataset and credits dataset .

3. PROJECT OBJECTIVE

Demonstrate Practical Implementation: Showcase the application of machine learning algorithms (content-based filtering) in real-world scenarios.

Enhance User Experience: Improve user satisfaction by providing personalized movie recommendations tailored to individual preferences.

Explore Data Analysis Techniques: Utilize exploratory data analysis to understand movie dataset characteristics and user preferences.

Evaluate Recommendation Algorithms: Assess the performance and effectiveness of content-based filtering methods through measures like accuracy and coverage.

Encourage Further Research: Provide a foundation for extending the project to other recommendation domains (e.g., music, books) or integrating advanced techniques such as deep learning for improved recommendation accuracy.

4. METHODOLOGY

PROJECT SETUP AND PLANNING :

Select Data Sources: We've identified suitable datasets for movies, including attributes like title, genre, cast, director, and user ratings from popular sources include IMDb and Kaggle datasets.

DATA COLLECTION AND PREPROCESSING :

Data Collection: We have gathered movie data from selected sources, ensuring data integrity and completeness.

Data Cleaning: We've performed preprocessing tasks such as handling missing values, null values and standardizing data formats.

Exploratory Data Analysis (EDA): We utilized Pandas to analyze distributions of numerical features like durations.

Processing: We have used techniques like TF-IDF to extract meaningful features from textual data such as movie descriptions or actor names.

Root word extraction: We've used porterstemmer to extract root verbs of each word in the overview for further efficiency in content based filtering, separating each word by comma and hence later

converting them to numerical forms(vectors/array) for better comparison

Vectorization: Convert text-based features into numerical vectors that machine learning algorithms can process effectively.

Feature Representation: Attribute Selection: We've chosen relevant attributes (e.g., genres, directors, actors) as features to represent movies.

Similarity Calculation: Compute similarity between movies based on feature vectors, using metrics such as cosine similarity

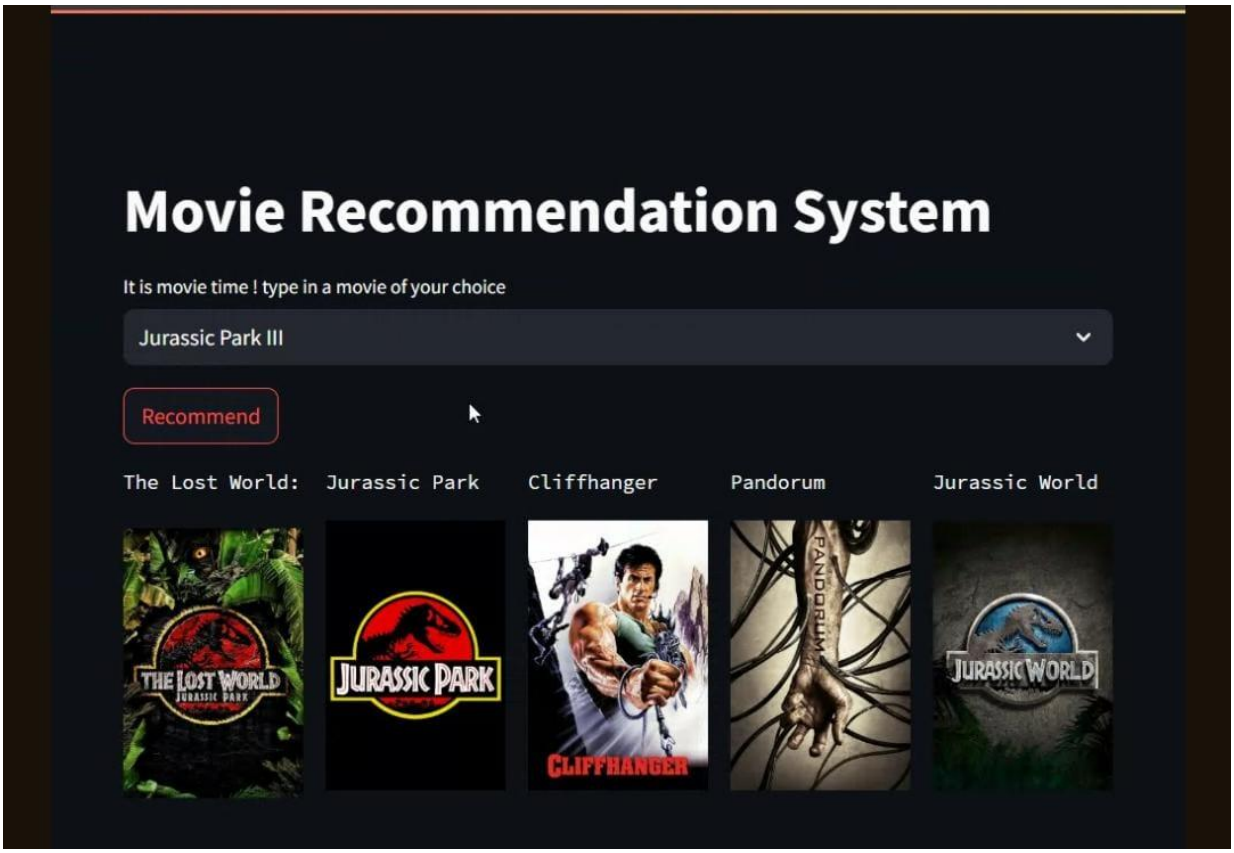
Cosine Similarity: Compute similarities between users or items based on their rating patterns using cosine similarity, measuring the cosine of the angle between the vectors.

Feature Vectorization: Convert categorical attributes into numerical feature vectors using cosine similarity.

Tools and Methods Used Python Libraries: Pandas, NumPy, Scikit-learn for data manipulation and machine learning algorithms; Streamlit for web interface.

Machine Learning Techniques: Content-based filtering, matrix factorization

Web app interface: We've used pycharm and streamlit for the process. Besides writing the code we've generated an api key to link the posters when recommending the movies in the interface.



5. DATA ANALYSIS AND RESULTS

We've extensively used various python libraries as previously mentioned, in addition to that we've created a subset of dataset based on the project requirements. We've carefully given in to the project demands and worked with the data the project required, eliminating any excess data for that manner again by using various python functionalities.

6. CONCLUSION

The movie recommendation system project successfully demonstrated the potential of data-driven solutions in providing personalized user experiences. By using content filtering techniques and a user-friendly web interface, the project has created a robust foundation for further development and innovation in the field of recommendation systems. Continuous improvement and adaptation to user feedback will ensure that the system remains relevant and effective in meeting user needs.

7. APPENDICES

THE EXTRACTED CODE IS:

```
import pandas as pd

import numpy as np

data = pd.read_csv('movies.csv')

data.head(10)

data.shape

data.info()

data.columns
```

```
movies=data[[      'id', 'genres' , 'keywords', 'original_language','overview',
            'original_title', 'popularity', 'production_companies', 'release_date', 'runtime',
            'title'        ]]
movies
movies.info()
movies.isnull().sum()
movies.dropna(axis=0, how= 'any')
df = pd.read_csv('credits.csv')
df.shape
df.info()
df.isnull().sum()
movies = movies.merge(df, on = 'title')
movies
movies.iloc[0:4801].genres
import ast
def convert(obj):
      L=[]
      for i in ast.literal_eval(obj):
            L.append(i['name'])
      return L
movies['genres'] = movies['genres'].apply(convert)
movies['keywords'] = movies['keywords'].apply(convert)
movies['production_companies'] = movies['production_companies'].apply(convert)
movies.head()
def convert3(obj):
```

```
    L=[]
    counter = 0
    for i in ast.literal_eval(obj):
        if counter != 3:
            L.append(i['name'])
            counter += 1
        else:
            break
        return L
movies['cast'] = movies['cast'].apply(convert3)
movies
def fetch_director(obj):
    L = []
    for i in ast.literal_eval(obj):
        if i['job']=='Director':
            L.append(i['name'])
        return L
movies['crew'] = movies['crew'].apply(fetch_director)
movies
movies['overview'][0]
def safe_split(x):
    if isinstance(x, str):
        return x.split()
    else:
        return []
```

```
movies['overview'] = movies['overview'].apply(safe_split)

movies

movies.isnull().sum()

movies = movies.dropna()

movies.isnull().sum()

movies['genres'] = movies['genres'].apply(lambda x:[i.replace(" ","") for i in x])

movies

movies['overview'] = movies['overview'].apply(lambda x:[i.replace(" , ","") for i in x])

movies

movies['overview'][0]

movies['keywords'] = movies['keywords'].apply(lambda x:[i.replace(" ","") for i in x])

movies['production_companies'] = movies['production_companies'].apply(lambda x:[i.replace(" ","") for
i in x])

movies

movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] + movies['cast'] +
movies['crew']

movies

new_df = movies[['id','title','tags']]

new_df

new_df['tags'] = new_df['tags'].apply(lambda x:" ".join(x)if isinstance(x, list) else str(x))

new_df

new_df['tags'][0]

new_df['tags'] = new_df['tags'].apply(lambda x:x.lower())

new_df

from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(max_features=5000,stop_words='english')
```

```
cv.fit_transform(new_df['tags']).toarray().shape

vectors = cv.fit_transform(new_df['tags']).toarray()

vectors[0]

len(cv.get_feature_names_out())

import nltk

from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

def stem(text):
    y = []
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)

new_df['tags'] = new_df['tags'].apply(stem)

from sklearn.metrics.pairwise import cosine_similarity

cosine_similarity(vectors)

cosine_similarity(vectors).shape

similarity = cosine_similarity(vectors)

similarity[0]

sorted(list(enumerate(similarity[0])),reverse=True,key=lambda x:x[1])[1:6]

def recommend(movie):
    movie_index = new_df[new_df['title'] == movie].index[0]
    dictance=similarity[movie_index]
    movies_list = sorted(list(enumerate(dictance)),reverse=True,key=lambda x:x[1])[1:6]
    for i in movies_list:
        print(new_df.iloc[i[0]].title)
```

```
def recommend(movie):
    movie_index = new_df[new_df['title'] == movie].index
    if movie_index.empty:
        print(f"Movie '{movie}' not found in the dataset.")
        return

    movie_index = movie_index[0]
    distance = similarity[movie_index]
    movies_list = sorted(list(enumerate(distance)), reverse=True, key=lambda x: x[1])[1:6]
    for i in movies_list:
        print(new_df.iloc[i[0]].title)

recommend('Jurassic Park')
recommend('Tangled')
recommend('Thor')
```

```
recommend('Jurassic Park')
```

```
Jurassic World
The Lost World: Jurassic Park
Jurassic Park III
The Nut Job
Jungle Shuffle
```

```
recommend('Tangled')
```

```
Out of Inferno
Aladdin
Toy Story 3
The Princess and the Frog
Frozen
```

```
recommend('Thor')
```

```
Thor: The Dark World
Captain America: Civil War
Captain America: The First Avenger
Man of Steel
Iron Man 3
```

DATASETS :

https://drive.google.com/drive/folders/10P7C-suaBAA5_ZAi4oEO_Pmsb7kG6JMZ

WEB INTERFACE CODE :

```
import streamlit as st
import pickle
import pandas as pd
import requests

def fetch_poster(movie_id):
    response =
requests.get('https://api.themoviedb.org/3/movie/{}?api_key=99eabbbc0eca91b6739ba194a393d1bf&language=
en-US'.format(movie_id))
    data = response.json()
    return "https://image.tmdb.org/t/p/w500/"+data['poster_path']

def recommend(movie):
    movie_index = movies[movies['title'] == movie].index[0]
    dictance = similarity[movie_index]
    movies_list = sorted(list(enumerate(dictance)), reverse=True, key=lambda x: x[1])[1:6]
    recommended_movies = []
    recommended_movies_posters = []
    for i in movies_list:
        movie_id = movies.iloc[i[0]].id

        recommended_movies.append(movies.iloc[i[0]].title)
        recommended_movies_posters.append(fetch_poster(movie_id))
    return recommended_movies,recommended_movies_posters

movies_dict = pickle.load(open('movie_dict.pkl','rb'))
movies = pd.DataFrame(movies_dict)
similarity = pickle.load(open('similarity.pkl','rb'))

st.title('Movie Recommendation System')
```

```
option = st.selectbox(
'It is movie time ! type in a movie of your choice',
movies['title'].values)

if st.button('Recommend'):
        names,posters = recommend(option)
        col1, col2, col3, col4, col5 = st.columns(5)
        with col1:
                st.text(names[0])
                st.image(posters[0])
        with col2:
                st.text(names[1])
                st.image(posters[1])
        with col3:
                st.text(names[2])
                st.image(posters[2])
        with col4:
                st.text(names[3])
                st.image(posters[3])
        with col5:
                st.text(names[4])
                st.image(posters[4])
```

REFERENCE :

https://www.geeksforgeeks.org/a-beginners-guide-to-streamlit/amp/

https://www.learndatasci.com/glossary/cosine-similarity/

https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/

https://youtu.be/HOMXxrSrNuw?si=T06poXlWdFC3RRrD